

Protocolos estándar y servicios REST

1. Protocolos estándar y servicios REST: leyendo el controlador

Protocolos estándar y servicios REST: leyendo el controlador

Dos programas pueden hablarse por red usando una IP y un puerto, y para entenderse necesitan un protocolo: un conjunto de reglas fijas sobre cómo tiene que ser cada mensaje, que los dos lados conocen de antemano — como rellenar un formulario con casillas fijas en vez de escribir una carta libre: quien lo lee sabe exactamente dónde va a encontrar cada dato.

Hoy vas a leer un controlador REST ya hecho, del mismo tipo que vas a construir tú mismo la semana que viene.

La URL, pieza a pieza

Esa petición viaja hacia una dirección concreta, la URL, que en una API web añade una pieza nueva sobre el host y el puerto que ya conoces: la ruta del recurso.

```
http://localhost:8080/api/v1/libros/3
```

protocolo host:puerto ruta

Las tres piezas de la URL

- Protocolo (http): las reglas de comunicación que van a usar los dos — la que vas a mirar por dentro en este apartado.
- Host y puerto (localhost:8080): dónde está el servidor y por qué puerta escucha.
- Ruta (/api/v1/libros/3): qué recurso concreto se pide dentro de ese servidor — esta pieza es la que vas a usar para identificar cada libro, cada usuario o cada pedido de tu API: uno por URL, sin ambigüedad.

Qué es HTTP

HTTP (HyperText Transfer Protocol) es el protocolo de petición-respuesta que usa la web: el cliente manda una petición con un formato fijo, el servidor responde con una respuesta con otro formato fijo, y ahí termina esa conversación — la siguiente petición empieza de cero.

Una petición HTTP real es texto plano

Por debajo del navegador, o de una herramienta como curl —la que usarás en la Actividad 1.1 para mandar peticiones a mano, sin escribir código—, una petición HTTP tiene esta forma:

```
GET /api/v1/libros/3 HTTP/1.1  
Host: localhost:8080  
Accept: application/json
```

Y la respuesta, también texto plano

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id": 3, "titulo": "El nombre del viento", "precio": 19.95}
```

Las mismas tres partes, en los dos sentidos

Parte	En la petición	En la respuesta
Primera línea	Método + ruta + versión	Versión + código de estado
Cabeceras (headers)	Metadatos: qué formato aceptas, quién eres...	Metadatos: qué formato devuelve, tamaño...
Cuerpo (body)	Datos que envías (puede no haberlo)	Datos que devuelve (puede no haberlo)

Accept y Content-Type no son lo mismo

Accept la manda el cliente: "esto es lo que quiero que me devuelvas". Content-Type la manda quien envía un cuerpo (aquí, el servidor en la respuesta): "esto es lo que de verdad te estoy mandando". Es como pedir un plato en un restaurante — le dices al camarero qué quieres (Accept), y cuando te lo trae, la etiqueta del plato confirma qué es realmente lo que tienes delante (Content-Type).

Un mismo servidor podría responder en JSON o en otro formato según lo que le pida cada cliente en su Accept; y Content-Type le dice a quien recibe el mensaje cómo tiene que leer ese cuerpo.

Cada petición declara un verbo

Verbo	Qué expresa	¿Repetirla produce el mismo efecto?
GET	Leer un recurso, sin modificarlo.	Sí — no cambia los datos del servidor.
POST	Crear un recurso nuevo.	No — repetirla puede crear otro recurso.
PUT	Reemplazar por completo un recurso existente.	Sí — deja el mismo resultado final.
PATCH	Modificar solo algunos campos.	Depende de cómo se defina la modificación.
DELETE	Eliminar un recurso.	Sí — el recurso queda eliminado.

Esa última columna tiene nombre: idempotencia

Una petición es idempotente cuando repetirla varias veces produce el mismo efecto final sobre los datos que ejecutarla una sola vez.

Piensa en el botón de llamada de un ascensor: pulsarlo una vez o pulsarlo nerviosamente diez veces no hace que lleguen diez ascensores. El efecto final es el mismo — el ascensor queda avisado de que tiene que ir a tu planta.

PUT y DELETE son idempotentes

PUT funciona así: reemplazar cinco veces un libro por los mismos datos deja el recurso exactamente igual que reemplazarlo una sola vez. DELETE también es idempotente: la primera petición elimina el libro y las siguientes lo dejan igualmente eliminado.

Eso no significa que todas las respuestas tengan que ser idénticas. El primer DELETE podría responder 204 No Content y el segundo 404 Not Found, porque el libro ya no existe. Las respuestas son distintas, pero el efecto final sobre los datos es el mismo: el recurso sigue eliminado.

POST, en cambio, normalmente no lo es

Es más parecido a pulsar varias veces el botón de compra de una máquina expendedora: cada pulsación puede crear un pedido nuevo, cobrarte otra vez o insertar otro recurso.

Con PATCH, depende de la modificación concreta

- «Pon el precio en 19,95 €» es idempotente: repetirlo deja siempre el mismo precio.
- «Aumenta el precio en 5 €» no lo es: cada repetición vuelve a sumar otros 5 €.

Una novedad del estándar: el método QUERY

En junio de 2026 se incorporó al estándar HTTP el método QUERY, pensado para búsquedas cuyos criterios son demasiado complejos o extensos para expresarlos cómodamente mediante parámetros en la URL. En esos casos, el cliente puede describir la consulta mediante un cuerpo, normalmente en formato JSON.

⚠ Spring todavía no lo soporta

Spring todavía no incorpora una anotación `@QueryMapping` ni permite seleccionar QUERY mediante `RequestMethod.QUERY`. Por ese motivo, en este curso conocerás para qué sirve, pero las consultas se implementarán mediante GET y parámetros en la URL, que Spring sí soporta directamente.

El servidor responde con un código de estado

Familia	Significa	Ejemplos
2xx	Todo ha ido bien	200 OK, 201 Created, 204 No Content
4xx	Error del cliente	400 Bad Request, 401/403, 404 Not Found
5xx	Error del servidor	—

No hace falta memorizar todos los códigos

Con estos 5-6 tienes cubierto casi todo lo que verás en el curso: 200 (lectura correcta), 201 (se ha creado algo), 204 (correcto, pero no hay nada que devolver — típico en un DELETE), 400 (petición mal formada), 401/403 (no autenticado / sin permiso — Tema 2), 404 (el recurso no existe).

JSON como formato del cuerpo

Es el formato en el que casi todas las APIs modernas envían y reciben datos: pares clave-valor, anidables, sin tipos estrictos declarados:

```
{  
  "titulo": "El nombre del viento",  
  "precio": 19.95,  
  "editorial": {  
    "nombre": "Plaza & Janés"  
  }  
}
```

Volverás a JSON con más detalle al construir cuerpos de petición en la Actividad 1.1 — de momento basta con reconocer la forma.

Qué es una API

Piensa en el mando de una tele: tiene un botón para subir volumen, otro para cambiar de canal, otro para apagar. No necesitas saber nada de electrónica para usarlo — solo conoces los botones disponibles y qué hace cada uno.

Una API (Application Programming Interface) es exactamente eso, pero para programas: el conjunto de operaciones que una aplicación expone para que otros programas la usen, sin que necesiten conocer cómo está construida por dentro.

Qué es REST

REST es un estilo concreto de diseñar esos "botones" sobre HTTP: los datos se modelan como recursos (un libro, un usuario, un pedido), cada recurso tiene su propia URL — su propio "botón" — y las operaciones sobre ese recurso se expresan con los verbos HTTP (GET para leerlo, POST para crearlo...).

Una API de librería, como ejemplo de patrón

Operación	Verbo + ruta
Listar todos los libros	GET /libros
Ver un libro concreto	GET /libros/3
Añadir un libro nuevo	POST /libros
Reemplazar un libro	PUT /libros/3
Borrar un libro	DELETE /libros/3

La ruta identifica qué recurso, y el verbo identifica qué operación — no hace falta una ruta distinta para cada acción (/libros/borrar/3 sería el estilo antiguo, no REST).

Leyendo un controlador REST completo

Con esa base, ya puedes leer un controlador REST real — de momento solo con los métodos GET (los de escritura llegan en el próximo apartado). Mira primero la forma general y luego ve bajando a la tabla, anotación por anotación.

El controlador completo

```
@RestController
@RequestMapping("/api/v1/libros")
@RequiredArgsConstructor
public class LibroController {
    private final LibroService libroService;

    @GetMapping
    public ResponseEntity<List<LibroResponseDTO>> getAll() {
        return ResponseEntity.ok(libroService.findAll());
    }

    @GetMapping("/{id}")
    public ResponseEntity<LibroResponseDTO> getById(@PathVariable Long id) {
        return ResponseEntity.ok(libroService.findById(id));
    }
}
```

El controller no consulta nada directamente

`getAll` y `getById` se limitan a llamar a `libroService.findAll()` y `libroService.findById(id)` y a envolver lo que les devuelve en una respuesta HTTP — es `LibroService` quien de verdad hace el trabajo. El controller no sabe nada de bases de datos: solo traduce entre HTTP y esa llamada Java.

Anotación a anotación, mirando solo la parte de HTTP

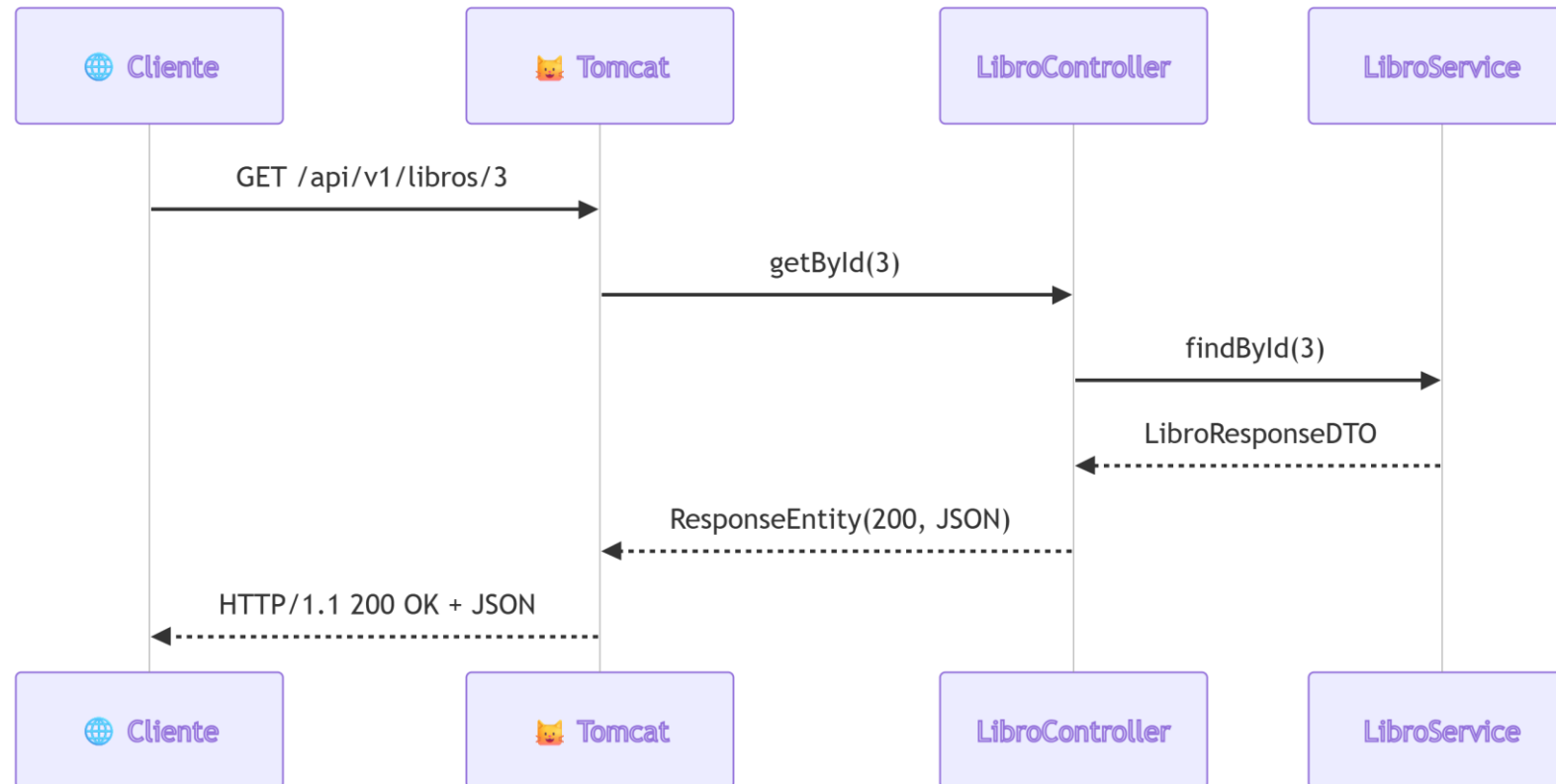
Anotación / elemento	Qué representa en HTTP
@RestController	Marca la clase como controlador cuyas respuestas se serializan a JSON directamente al cuerpo HTTP.
@RequestMapping("/api/v1/libros")	La ruta base del recurso. El /v1 es el versionado: se puede publicar un /v2 sin romper a los clientes de la v1.
@GetMapping / @GetMapping("/{id}")	Verbo (GET) + ruta = una operación concreta. Spring elige uno de los dos según cómo termine la ruta.
@PathVariable Long id	Extrae el trozo variable de la URL y lo entrega como parámetro Java, ya convertido a Long.
ResponseEntity.ok(...)	Construye la respuesta con el código 200 explícito y el cuerpo indicado.

Nada de esto es magia añadida

Vuelve a GET /api/v1/libros/3 HTTP/1.1. Ese GET, esa ruta y ese 3 son exactamente lo que decide Spring para escoger getByld y rellenar su parámetro id — es la traducción directa de esas tres piezas del texto plano a una llamada Java.

El viaje completo, ida y vuelta

Fíjate en que la respuesta recorre exactamente el mismo camino que la petición, pero al revés:



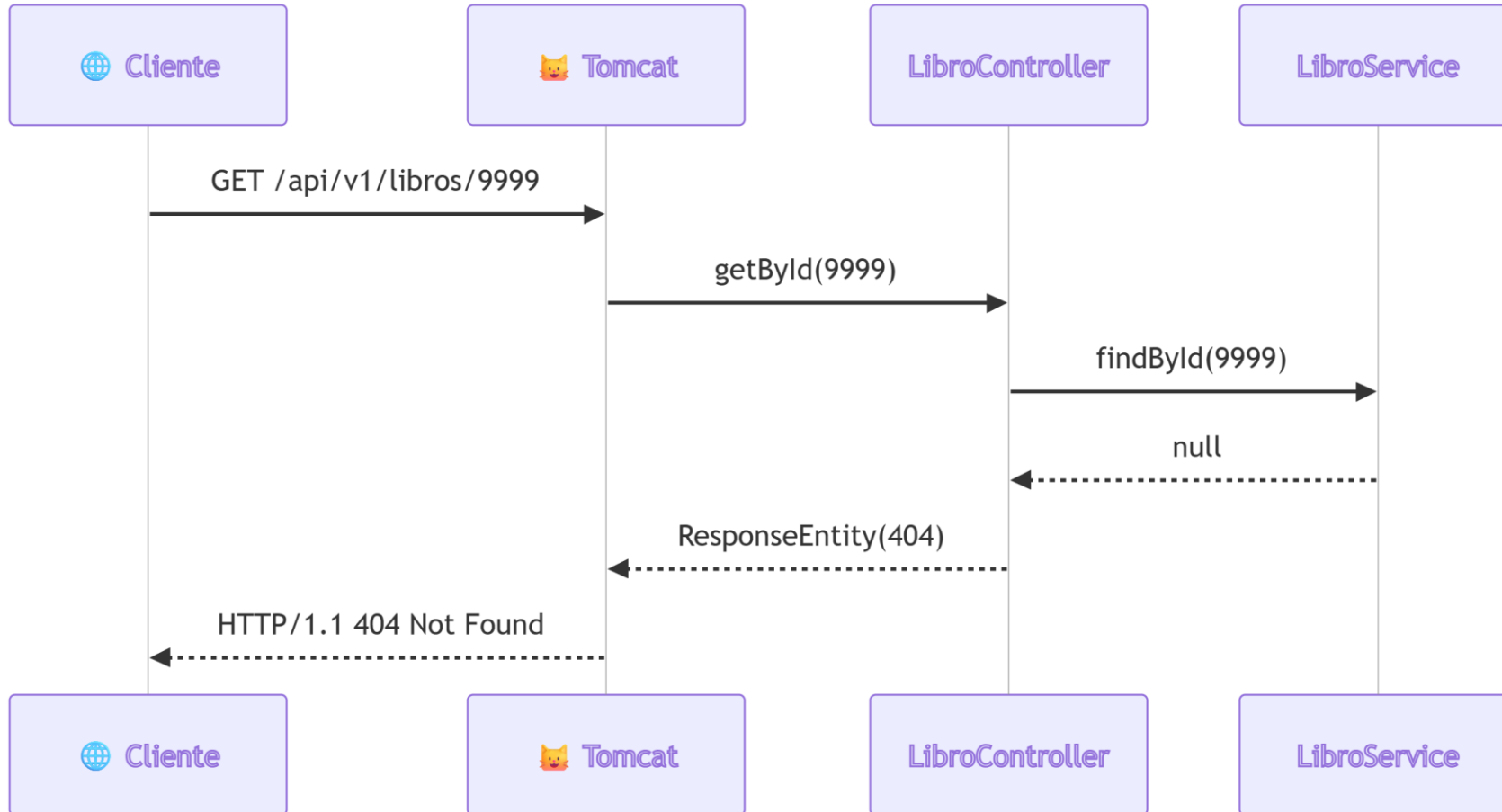
El puerto 8080 y Tomcat no son magia

El servidor Tomcat que escucha en el puerto 8080 lo trae la dependencia `spring-boot-starter-webmvc` del `pom.xml` — es la "librería que implementa el servicio en red" de la que habla el currículum. Tú no arrancas ningún servidor a mano: Spring Boot lo hace por ti al ejecutar la clase anotada con `@SpringBootApplication`.

¿Y si el libro no existe?

Todo lo anterior supone que existe un libro con el id solicitado. Cuando no existe, el recorrido de la petición es casi el mismo.

El mismo viaje, pero sin libro al final



El controller comprueba el resultado y lo traduce

En esta primera versión, el service intenta localizar el libro y devuelve null cuando no lo encuentra:

```
@GetMapping("/{id}")
public ResponseEntity<LibroResponseDTO> getById(@PathVariable Long id) {
    LibroResponseDTO libro = libroService.findById(id);

    if (libro == null) {
        return ResponseEntity.notFound().build();
    }

    return ResponseEntity.ok(libro);
}
```

Una respuesta distinta según el resultado

- Si existe el libro, devuelve 200 OK con el libro en el cuerpo.
- Si no existe, devuelve 404 Not Found sin cuerpo.

En una línea, cada caso

```
libro encontrado    → ResponseEntity.ok(libro)        → 200  
libro no encontrado → ResponseEntity.notFound()      → 404
```

Esta versión con if es solo el primer paso

Esta versión con if permite ver claramente cómo una situación de la aplicación se convierte en un código HTTP. Más adelante utilizarás una forma más limpia: el service lanzará directamente una excepción mediante `orElseThrow()`, y el controller no necesitará repetir esta comprobación.

Por qué un protocolo estándar

Podrías diseñar tu propio protocolo casero sobre sockets (Tema 4) en vez de usar HTTP/REST. La diferencia es que HTTP es un protocolo estándar: cualquier cliente que exista ya sabe hablarlo, sin que tengas que documentar ni acordar nada a medida. Con un protocolo propio, cada cliente nuevo tendría que aprender tus reglas particulares desde cero.

Estándar frente a protocolo a medida

	Protocolo estándar (HTTP)	Protocolo propio a medida
¿Quién ya sabe hablarlo?	Cualquier cliente existente: navegadores, curl, librerías de cualquier lenguaje.	Solo el que tú mismo escribas para hablarlo.
Documentación necesaria	La del propio recurso — el formato de mensaje ya está definido.	Tienes que documentar también el propio formato de mensaje.
Herramientas de por medio	Servidores, proxies, balanceadores, cachés — todos entienden HTTP.	Ninguna herramienta genérica te sirve; hay que escribirlas todas.
Coste de un cliente nuevo	Bajo: ya sabe HTTP, solo aprende tus rutas.	Alto: tiene que aprender tu protocolo entero desde cero.

Ideas clave

- Toda comunicación HTTP es petición-respuesta entre un cliente y un servidor, con la URL como dirección (protocolo, host, puerto, ruta).
- HTTP es texto con una primera línea, cabeceras y cuerpo opcional; Accept es lo que pide el cliente, Content-Type es lo que confirma el que envía el cuerpo.
- Los verbos HTTP expresan la intención: GET lee, POST crea, PUT reemplaza, PATCH modifica parcialmente y DELETE borra.
- Los códigos de estado (2xx/4xx/5xx) resumen el resultado sin tener que leer el cuerpo.
- JSON es el formato habitual del cuerpo de petición/respuesta en las APIs modernas.

Ideas clave (cont.)

- REST modela los datos como recursos con URL propia, y usa los verbos HTTP para operar sobre ellos — la ruta identifica qué, el verbo identifica qué operación.
- `@RestController` + `@RequestMapping` + `@GetMapping` traducen "verbo + ruta" en un método Java concreto; `ResponseEntity` construye la respuesta con su código de estado.
- Un código como 404 no aparece automáticamente: la aplicación detecta que el recurso no existe y lo traduce a una respuesta HTTP.
- El servidor embebido (Tomcat) lo trae `spring-boot-starter-webmvc` — no lo arrancas tú a mano.